1️⃣ What is an Abstract Class?

An **abstract class** is a class that **cannot be instantiated (no objects can be created directly)**.
It is meant to be used **as a base class for other classes**.

It may contain:

- **Abstract methods** (no implementation)
- **Normal methods** (with implementation)
- **Fields**
- **Properties**
- **Constructors**

Definition

An abstract class is a base class that provides a common structure for derived classes and may contain abstract methods that must be implemented by child classes.

2️⃣ Why Do We Use Abstract Classes?

Abstract classes are used when:

- Multiple classes share **common behavior**
- Some behavior must be **implemented by child classes**

Example:

Hospital system

All staff may have:

- ID
- Name
- Work()

But **Doctors**, **Nurses**, **Technicians** work differently.

So we define a **common structure in the abstract class**.

3️⃣ Syntax of Abstract Class

```
abstract class ClassName
{
   // abstract and non-abstract members
}
```

Example:

```
abstract class Animal
{
   public abstract void MakeSound();
}
```

4️⃣ Abstract Method

An **abstract method**:

- Has **no implementation**
- Must be implemented in the **derived class**
- Must be declared using **abstract**

Example:

```
abstract class Animal
{
   public abstract void MakeSound();
}
```

Notice:

```
public abstract void MakeSound();
```

There is **no method body**.

---

5 Derived Class Implementing Abstract Method

When a class inherits from an abstract class, it **must implement all abstract methods** using override.

Example:

Abstract Base Class

```
using System;


abstract class Animal
{
    public abstract void MakeSound();
}
```

Derived Class

```
class Dog : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Dog barks");
    }
}
```

Main Program

```
class Program
{
    static void Main()
    {
        Dog d = new Dog();
        d.MakeSound();
```

```
    }
}
```

Output

Dog barks

6️⃣ Abstract Class Cannot Be Instantiated

This is a **very important rule**.

❌ Not allowed:

```
Animal a = new Animal();
```

Error:

Cannot create an instance of abstract class

But this is allowed:

```
Animal a = new Dog();
```

This is **polymorphism**.

7️⃣ Abstract Class Can Have Normal Methods

Abstract classes can contain **fully implemented methods**.

Example:

```
abstract class Animal
{
    public void Eat()
    {
        Console.WriteLine("Animal is eating");
    }

    public abstract void MakeSound();
```

```
}
```

Derived class:

```
class Dog : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Dog barks");
    }
}
```

Usage:

```
Dog d = new Dog();
d.Eat();
d.MakeSound();
```

Output

```
Animal is eating
Dog barks
```

8️⃣ Abstract Class Can Have Properties

Example:

```
abstract class Person
{
    public string Name { get; set; }

    public abstract void Work();
}
```

Derived class:

```
class Doctor : Person
{
    public override void Work()
    {
        Console.WriteLine(Name + " treating patients");
    }
}
```

Usage:

```
Doctor d = new Doctor();
d.Name = "Dr. John";
d.Work();
```

Output

```
Dr. John treating patients
```

9️⃣ Abstract Class Can Have Constructors

Abstract classes **can contain constructors**.

They are called when **derived class objects are created**.

Example:

```
abstract class Person
{
    public Person()
    {
        Console.WriteLine("Person constructor called");
    }
}
```

Derived class:

```
class Employee : Person
{
    public Employee()
    {
        Console.WriteLine("Employee constructor called");
    }
}
```

Usage:

```
Employee e = new Employee();
```

Output

```
Person constructor called
Employee constructor called
```

🔟 Real-world Example (Healthcare Domain)

Abstract base class:

```
abstract class HospitalStaff
{
    public string Name { get; set; }

    public void Login()
    {
        Console.WriteLine(Name + " logged in");
    }

    public abstract void PerformDuty();
}
```

Doctor class:

```
class Doctor : HospitalStaff
{
    public override void PerformDuty()
    {
        Console.WriteLine("Doctor treating patients");
    }
}
```

Nurse class:

```
class Nurse : HospitalStaff
{
    public override void PerformDuty()
    {
        Console.WriteLine("Nurse assisting doctor");
    }
}
```

Usage:

```
Doctor d = new Doctor();
d.Name = "Dr. Smith";

d.Login();
d.PerformDuty();
```

Output

```
Dr. Smith logged in
Doctor treating patients
```

---

1 1 Rules of Abstract Classes

Rule	Description
Cannot create object	Yes
Can contain abstract methods	Yes
Can contain normal methods	Yes
Must be inherited	Yes
Child class must implement abstract methods	Yes